

Gebze Technical University
CSE 341 — Concepts of Programming Languages · Spring 2026
Project Part 1 — Example Programs & Test Report (D3)

ScoreScript

Student Number: 230104004090
Name: FATİH EMRE OĞAN
Submission Date: 8 May 2026

This report documents the example programs and test cases for the ScoreScript Part 1 submission. Part A presents three valid programs that together exercise every language construct. Part B presents five malformed programs, each triggering a distinct parser error with a line number.

All programs were tested using: `java -cp out Main <file>`

Part A — Valid Programs

The three programs below together cover every construct in the Part 1 grammar: student declarations, group declarations, variable declarations, function declarations with if/else chains, for loops, all four domain operations (curve, average, rank, transcript), assignment statements, print statements, and expressions with multiple precedence levels.

Program 1 — program1.ss

Focus: Student declarations, group declarations, variable declarations, all four domain operations

Constructs demonstrated: student, group, int variable, curve, average, rank, transcript

Source Code

```
student s1 {
  id: "230104004090",
  firstName: "Fatih Emre",
  lastName: "OGAN",
  grades: { math: 85, science: 90, history: 72 }
}
student s2 {
  id: "230104004091",
  firstName: "Ayse",
  lastName: "KURT",
  grades: { math: 91, science: 88, history: 95 }
}
student s3 {
  id: "230104004092",
  firstName: "Mehmet",
  lastName: "DEMIR",
  grades: { math: 60, science: 70, history: 65 }
}
student s4 {
  id: "230104004093",
  firstName: "Zeynep",
  lastName: "ARSLAN",
  grades: { math: 45, science: 55, history: 50 }
}
group ClassZ23 = [s1, s2, s3, s4]
```

```

group TopStudents = [s1, s2]
int curveAmount = 5
int passingThreshold = 60
curve ClassZ23 by curveAmount
average ClassZ23
rank ClassZ23
transcript s1
transcript s2
transcript s3
transcript s4
average TopStudents
rank TopStudents

```

Parser Output

```
java -cp out Main examples/valid/program1.ss
```

```
OK ? 'examples/valid/program1.ss' parsed successfully.
```

Program 2 — program2.ss

Focus: User-defined functions with if/else if/else chains, bool variables, print statements

Constructs demonstrated: func, if/else if/else, bool variable, float variable, print, call statement, assignment

Source Code

```

student s1 { id: "230104004090", firstName: "Fatih Emre", lastName: "OGAN",
  grades: { math: 85, science: 90, english: 78 } }
student s2 { id: "230104004094", firstName: "Selin", lastName: "YILMAZ",
  grades: { math: 42, science: 55, english: 60 } }
student s3 { id: "230104004095", firstName: "Burak", lastName: "SAHIN",
  grades: { math: 95, science: 98, english: 92 } }
group SectionA = [s1, s2, s3]
float passingGrade = 59.5
float honorThreshold = 90.0
bool curveApplied = false
func letterGrade(avg: float) -> string {
  if avg >= 90 { return "AA" }
  else if avg >= 85 { return "BA" }
  else if avg >= 75 { return "BB" }
  else if avg >= 65 { return "CB" }
  else if avg >= 60 { return "CC" }
  else { return "FF" }
}
func isPassing(avg: float) -> bool {
  if avg > passingGrade { return true }
  else { return false }
}
func printLabel(avg: float) -> void {
  if avg >= honorThreshold { print "Status: HONOR STUDENT" }
  else if avg >= passingGrade { print "Status: PASSING" }
  else { print "Status: FAILING" }
}
transcript s1
transcript s2
transcript s3

```

```

average SectionA
if curveApplied == false {
    curve SectionA by 3
    curveApplied = true
}
average SectionA
rank SectionA
bool pass1 = isPassing(72.0)
string grade1 = letterGrade(88.5)
print pass1
print grade1
printLabel(92.0)
printLabel(45.0)

```

Parser Output

```
java -cp out Main examples/valid/program2.ss
```

```
OK ? 'examples/valid/program2.ss' parsed successfully.
```

Program 3 — program3.ss

Focus: For loops over groups, assignment statements, curve with computed expression, nested if inside for

Constructs demonstrated: for student ... in, assign_stmt, curve by <expr>, multiple groups, if inside for body

Source Code

```

student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90, history: 72 }
}

student s2 {
    id: "230104004091",
    firstName: "Ayse",
    lastName: "KURT",
    grades: { math: 91, science: 88, history: 95 }
}

student s3 {
    id: "230104004092",
    firstName: "Mehmet",
    lastName: "DEMIR",
    grades: { math: 60, science: 70, history: 65 }
}

student s4 {
    id: "230104004096",
    firstName: "Deniz",
    lastName: "KAPLAN",
    grades: { math: 30, science: 40, history: 35 }
}

student s5 {
    id: "230104004097",
    firstName: "Elif",
    lastName: "CELIK",

```

```

    grades: { math: 78, science: 82, history: 80 }
}

group MorningClass = [s1, s2, s3]
group AfternoonClass = [s4, s5]
group AllStudents = [s1, s2, s3, s4, s5]

int baseCurve = 4
int bonusCurve = 3
float lowAvgCutoff = 65.0

func needsCurve(avg: float) -> bool {
    if avg < lowAvgCutoff { return true }
    else { return false }
}

for student s in AllStudents {
    transcript s
}

average MorningClass
rank MorningClass
average AfternoonClass
rank AfternoonClass

curve MorningClass by baseCurve + bonusCurve
curve AfternoonClass by baseCurve

rank AllStudents

for student s in MorningClass {
    transcript s
    print needsCurve(lowAvgCutoff)
}

baseCurve = baseCurve + 1
bonusCurve = 2

if needsCurve(58.0) == true {
    curve AfternoonClass by baseCurve + bonusCurve
    average AfternoonClass
    rank AfternoonClass
}

for student s in AllStudents {
    transcript s
}

average AllStudents
rank AllStudents

```

Parser Output

```
java -cp out Main examples/valid/program3.ss
```

```
OK ? 'examples/valid/program3.ss' parsed successfully
```

Part B — Invalid Programs

The five programs below each contain a single deliberate error. Every error is caught at parse time and reported with the line number. The table below summarises all five cases.

File	Error Type	Line	Error Message
error1.ss	Wrong field order in student	10	expected 'firstName' but found 'lastName'
error2.ss	'.' instead of '=' in group	22	expected '=' but found '.'
error3.ss	Missing '->' in func	17	expected '->' but found 'string'
error4.ss	Missing expr after 'by'	19	expected an expression but found 'average'
error5.ss	Statement before declarations end	18	unexpected token 'student'

Error 1 — error1.ss

Error type: Wrong field order in student block — lastName before firstName

Error line: 10

Source Code

```
student s1 {
    id: "230104004090",
    lastName: "OGAN",
    firstName: "Fatih Emre",
    grades: { math: 85, science: 90 }
}

group ClassZ23 = [s1]

transcript s1
```

Parser Error Output

```
java -cp out Main examples/invalid/error1.ss
Parse error at line 10: expected 'firstName' but found 'lastName'
```

Explanation

The student declaration enforces fixed field order: id, firstName, lastName, grades. This program places lastName before firstName. The parser calls expect(KW_FIRST_NAME) after consuming id and immediately rejects the token lastName.

Error 2 — error2.ss

Error type: Colon used instead of equals sign in group declaration

Error line: 22

Source Code

```
student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90 }
}

student s2 {
    id: "230104004091",
    firstName: "Ayse",
    lastName: "KURT",
    grades: { math: 91, science: 88 }
}

group ClassZ23 : [s1, s2]

average ClassZ23
```

Parser Error Output

```
java -cp out Main examples/invalid/error2.ss
Parse error at line 22: expected '=' but found ':'
```

Explanation

The group declaration requires '=' between the group name and the member list. This program uses ':' instead. The parser calls expect(EQ) and rejects the colon token.

Error 3 — error3.ss

Error type: Missing arrow (->) in func declaration

Error line: 17

Source Code

```
student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90 }
}

group ClassZ23 = [s1]

func letterGrade(avg: float) string {
    if avg >= 90 { return "AA" }
    else if avg >= 85 { return "BA" }
    else { return "FF" }
}

transcript s1
```

Parser Error Output

```
java -cp out Main examples/invalid/error3.ss
Parse error at line 17: expected '->' but found 'string'
```

Explanation

Every function declaration requires '->' between the parameter list and the return type. This program omits the arrow. The parser calls expect(ARROW) after consuming ')' and rejects the type keyword 'string'.

Error 4 — error4.ss

Error type: Missing expression after 'by' in curve statement

Error line: 19

Source Code

```
student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90 }
}

group ClassZ23 = [s1]

curve ClassZ23 by

average ClassZ23
```

Parser Error Output

```
java -cp out Main examples/invalid/error4.ss
Parse error at line 19: expected an expression but found 'average'
```

Explanation

The curve operation requires a valid expression after 'by'. This program omits the expression. The parser enters parseExpr() and finds the keyword 'average' which is not a valid expression start.

Error 5 — error5.ss

Error type: Statement appears before declaration section ends

Error line: 18

Source Code

```
student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90 }
}

curve ClassZ23 by 5

student s2 {
    id: "230104004091",
    firstName: "Ayse",
```

```
    lastName: "KURT",  
    grades: { math: 91, science: 88 }  
}  
  
group ClassZ23 = [s1, s2]  
  
average ClassZ23
```

Parser Error Output

```
java -cp out Main examples/invalid/error5.ss
```

```
Parse error at line 18: unexpected token 'student' ? expected a statement
```

Explanation

ScoreScript requires all declarations to precede all statements. Once the parser sees 'curve' it enters the statement section permanently. It then finds 'student' — a declaration keyword — which is not a valid statement.